

FREDERICO LUIZ STREY

**DETECÇÃO DE RACHADURAS COM YOLO_{v8}
FINE-TUNING, INFERÊNCIA E MICROSERVIÇO
FASTAPI**

Relatório Técnico

Vitória - ES,
31 de maio de 2026

Sumário

1	Introdução	2
2	Fundamentação Teórica	3
2.1	Modelos YOLO	3
2.2	Transfer Learning e Fine-Tuning	3
2.3	Métricas de Avaliação	3
3	Metodologia	4
3.1	Configuração do Ambiente	4
3.2	Conjunto de Dados	4
3.3	Arquitetura do Modelo	4
3.4	Hiperparâmetros de Treinamento	5
3.5	Exportação do Modelo	6
4	Resultados	6
4.1	Métricas Quantitativas	6
5	Microserviço FastAPI	7
5.1	Arquitetura Geral	7
5.2	Módulo de Inferência	7
5.3	Endpoint de Predição	8
5.4	Endpoints Auxiliares	9
5.5	Interface Web	9
6	Empacotamento e Reprodutibilidade	10
7	Discussão	10
7.1	Justificativa do Fine-Tuning	10
7.2	Limitações	11
8	Conclusão	11

1 Introdução

A detecção e o monitoramento de fissuras em superfícies estruturais representam um problema clássico de visão computacional aplicada à engenharia civil e à manutenção predial. Inspeções manuais são custosas, sujeitas a erros humanos e de difícil escalonamento em ambientes amplos. Soluções baseadas em aprendizado profundo têm demonstrado capacidade de automatizar esse processo com precisão comparável ou superior à de especialistas humanos (1, 2).

Este relatório documenta o desenvolvimento de um sistema de detecção de rachaduras baseado na família de modelos YOLO (3), especificamente o YOLOv8 (4), utilizando a estratégia de *fine-tuning* sobre um conjunto de imagens rotuladas. O produto final compreende:

- um modelo treinado (**best.pt**) capaz de localizar e segmentar rachaduras;
- um microserviço HTTP construído com FastAPI (5) para servir inferências;
- uma interface web simples para demonstração do fluxo completo.

Treinar uma rede neural profunda desde o início exige uma grande quantidade de imagens rotuladas, elevado poder computacional e um tempo considerável de treinamento para que o modelo aprenda características visuais básicas, como bordas, texturas, contrastes e padrões geométricos.

Como o conjunto de dados disponibilizado para o desafio, bem como o tempo disponível para desenvolvimento da solução, eram relativamente reduzidos, a estratégia adotada foi utilizar um modelo pré-treinado da família YOLOv8 e realizar o processo de fine-tuning. Nessa abordagem, o conhecimento previamente adquirido durante o treinamento em grandes bases de dados é reaproveitado, permitindo que a rede já possua representações visuais genéricas e concentre o aprendizado apenas nas características específicas das rachaduras presentes nas imagens do projeto.

Além de reduzir significativamente o custo computacional e o tempo de treinamento, o fine-tuning tende a produzir modelos mais robustos quando há poucas amostras disponíveis, uma vez que aproveita informações aprendidas em milhões de imagens durante a fase de pré-treinamento. Dessa forma, foi possível obter um detector especializado para o problema proposto utilizando apenas as imagens fornecidas, sem a necessidade de construir uma arquitetura ou realizar um treinamento completo desde o início.

2 Fundamentação Teórica

2.1 Modelos YOLO

A família YOLO (*You Only Look Once*) foi introduzida por Redmon et al.(3) como uma abordagem de detecção de objetos em tempo real que reformula o problema como uma única passagem de regressão sobre a imagem (3). Ao contrário de métodos em dois estágios como o R-CNN (6), o YOLO processa a imagem inteira em uma única inferência, tornando-o significativamente mais rápido sem perda expressiva de qualidade.

O YOLOv8, desenvolvido pela Ultralytics (4), representa uma das evoluções mais recentes da família, com suporte nativo a tarefas de detecção, segmentação de instâncias e estimação de pose.

2.2 Transfer Learning e Fine-Tuning

Transfer learning é a prática de reutilizar os pesos de um modelo treinado em uma tarefa de origem para acelerar ou melhorar o aprendizado em uma tarefa alvo (7). O *fine-tuning* é uma forma específica de *transfer learning* em que todos os pesos do modelo (ou um subconjunto) são atualizados com uma taxa de aprendizado tipicamente baixa sobre os novos dados (8).

Essa abordagem é especialmente vantajosa quando:

- o conjunto de dados alvo é relativamente pequeno;
- o domínio visual da tarefa alvo guarda alguma similaridade com a tarefa de origem;
- o custo computacional de um treino completo é proibitivo.

No contexto de visão computacional para inspeção de infraestrutura, estudos recentes confirmam que modelos YOLO ajustados com *fine-tuning* superam arquiteturas treinadas do zero quando os dados disponíveis são limitados (9, 10).

2.3 Métricas de Avaliação

As métricas principais utilizadas para avaliação de detectores de objetos são:

- **Precisão (P):** fração das detecções corretas entre todas as detecções realizadas.
- **Recall (R):** fração das instâncias reais corretamente detectadas.
- **mAP@50:** *mean Average Precision* calculada com limiar de IoU de 0,50 (11).
- **mAP@50-95:** média do mAP para limiares de IoU entre 0,50 e 0,95 em passos de 0,05, adotada como métrica principal pelo COCO (12).

Para a tarefa de segmentação, as mesmas métricas são calculadas sobre as máscaras preditas em vez das *bounding boxes*.

3 Metodologia

3.1 Configuração do Ambiente

O treinamento foi executado na plataforma Kaggle, utilizando uma GPU NVIDIA Tesla T4 (14 GB de VRAM). O ambiente de software consistia em:

- Python 3.12.13
- PyTorch 2.10.0 com suporte CUDA 12.8
- Ultralytics 8.4.57

O notebook contendo todo o processo de treinamento, validação e geração dos resultados está disponível publicamente na plataforma Kaggle:

Notebook de Treinamento no Kaggle

3.2 Conjunto de Dados

O dataset foi organizado no formato exigido pelo Ultralytics YOLO, com imagens e anotações de segmentação de instâncias divididas em três partições: treino, validação e teste. A partição de teste continha **155 imagens** totalizando **200 instâncias** de rachaduras anotadas, sem imagens de fundo (*backgrounds*) e sem arquivos corrompidos, conforme relatado durante o processo de validação.

O arquivo de configuração `dataset.yaml` especificava os caminhos de cada partição e a lista de classes, composta por uma única classe: `rachadura`.

3.3 Arquitetura do Modelo

O modelo adotado neste trabalho foi o **YOLOv8s-seg** (*You Only Look Once v8 Small Segmentation*), uma variante compacta da família YOLOv8. A escolha dessa arquitetura ocorreu devido ao equilíbrio entre precisão, velocidade de inferência e custo computacional, tornando-a adequada para cenários com recursos limitados.

Durante o processo de *fine-tuning*, a arquitetura da rede permaneceu essencialmente a mesma, porém sua camada de saída foi automaticamente adaptada para o problema proposto. O modelo original foi treinado para reconhecer 80 categorias de objetos distintas, enquanto o conjunto de dados deste projeto continha apenas uma classe: *rachadura*. Por esse motivo, durante a inicialização do treinamento foi realizada a substituição automática

da configuração de classes ($nc=80$) por ($nc=1$), permitindo que a rede especializasse seus parâmetros exclusivamente na detecção e segmentação de rachaduras.

A versão final validada apresentou as seguintes características:

- 86 camadas fusadas para inferência;
- 11.779.987 parâmetros treináveis;
- aproximadamente 39,9 GFLOPs por inferência;
- suporte simultâneo à detecção e segmentação de instâncias.

Durante a validação final, o modelo alcançou desempenho satisfatório para o conjunto de dados disponível na tarefa de detecção de rachaduras, demonstrando que a estratégia de transferência de aprendizado foi suficiente para especializar a rede ao domínio específico do problema sem a necessidade de treinamento completo a partir do zero.

3.4 Hiperparâmetros de Treinamento

A Listagem 1 apresenta o código de treinamento utilizado.

```
1 from ultralytics import YOLO
2
3 model = YOLO(MODELO_BASE) # pesos pre-treinados (COCO)
4
5 results = model.train(
6     data=str(yaml_path),
7     epochs=EPOCHS,
8     imgsz=IMGSZ,
9     batch=BATCH,
10    patience=PATIENCE,
11    device=0,          # GPU 0
12    workers=2,
13    project='/kaggle/working/runs',
14    name='rachadura_seg',
15    exist_ok=True,
16    augment=True,      # augmentacao automatica
17    degrees=15.0,      # rotacao aleatoria +/- 15 graus
18    flipud=0.3,        # flip vertical 30% das vezes
19    mosaic=1.0,        # mosaico: combina 4 imagens em 1
20    close_mosaic=10,   # desliga mosaico nas ultimas 10 epocas
21    verbose=True,
22 )
```

Listing 1: Script de fine-tuning do YOLOv8s-seg.

A estratégia de *data augmentation* adotada inclui rotação aleatória, *flip* vertical e *mosaic augmentation* (9), técnica que combina quatro imagens em um único exemplo de treinamento, aumentando a variabilidade contextual sem exigir dados adicionais. O mosaico foi desativado nas últimas 10 épocas (`close_mosaic=10`) para permitir que o modelo refine seu aprendizado sobre imagens individuais ao final do treinamento.

A parâmetro `patience` ativa a parada antecipada (*early stopping*), interrompendo o treino se não houver melhora no mAP de validação por um número determinado de épocas consecutivas.

3.5 Exportação do Modelo

Após a validação, o modelo foi exportado para o formato ONNX (13) para portabilidade, dispensando a instalação do pacote `ultralytics` no ambiente de inferência:

```
1 model_eval.export(format='onnx', imgsz=IMGSZ, dynamic=True)
```

Listing 2: Exportação para ONNX.

4 Resultados

4.1 Métricas Quantitativas

A avaliação final foi conduzida sobre a partição de teste com os pesos do melhor *checkpoint* (`best.pt`). A Tabela 1 sumariza as métricas obtidas.

Tabela 1: Resultados de avaliação do YOLOv8s-seg na partição de teste.

Tarefa	Precisão (P)	Recall (R)	mAP@50	mAP@50-95
Detecção (<i>box</i>)	0,768	0,760	0,818	0,634
Segmentação (<i>mask</i>)	0,774	0,660	0,728	0,276

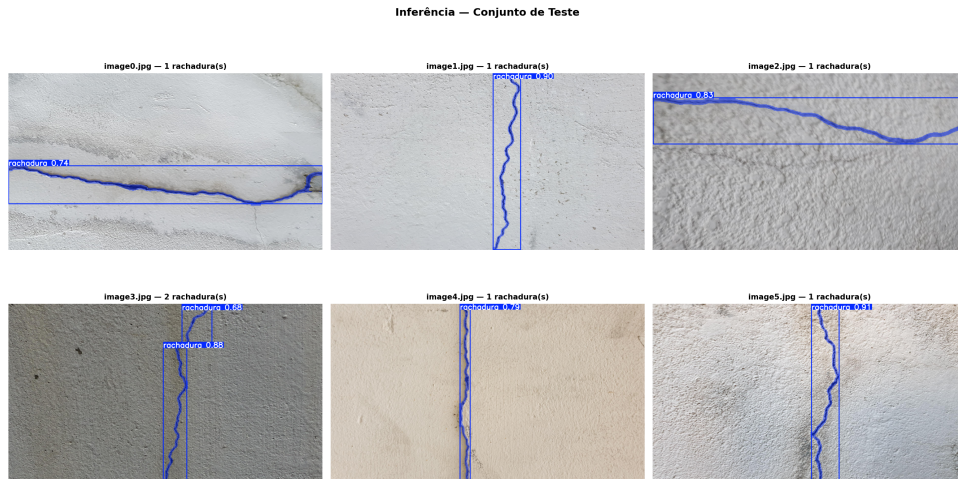


Figura 1: Imagens com as previsões do modelo.

Os resultados indicam um desempenho sólido para a tarefa de detecção com $mAP@50$ de **0,818**, valor consistente com detectores de estado da arte para conjuntos de dados de inspeção estrutural de porte similar (10, 1). A métrica $mAP@50-95$ para segmentação de máscaras (**0,276**) é mais conservadora, o que é esperado: localizar com precisão os contornos de fissuras finas exige máscaras muito precisas, e pequenos desvios de borda penalizam fortemente essa métrica nos limiares de IoU mais altos.

5 Microserviço FastAPI

5.1 Arquitetura Geral

O pipeline de inferência foi encapsulado em um microserviço HTTP desenvolvido com FastAPI (5), framework assíncrono de alto desempenho para Python.

A escolha do FastAPI é justificada por:

- suporte nativo a operações assíncronas (`async/await`);
- validação automática de esquemas com Pydantic (14);
- geração automática de documentação OpenAPI;
- baixa sobrecarga em comparação com frameworks síncronos como Flask (15).

5.2 Módulo de Inferência

A classe `CrackDetector` (Listagem 3) encapsula o modelo de segmentação e expõe um método `predict thread-safe` por meio de um mecanismo de sincronização baseado em `threading.Lock`. Essa abordagem evita condições de corrida (*race conditions*) quando múltiplas requisições são processadas simultaneamente pelo servidor, garantindo acesso

seguro aos recursos compartilhados do modelo durante a inferência. Além disso, o encapsulamento da lógica de predição em uma única classe facilita a portabilidade da solução para ambientes embarcados e dispositivos móveis, permitindo que o mesmo componente seja reutilizado tanto em arquiteturas de servidor quanto em aplicações executadas localmente em hardware com recursos computacionais limitados.

```
1 class CrackDetector:
2     def __init__(self, model_path: Path, confidence: float = 0.25):
3         self._model = YOLO(str(model_path))
4         self._lock = threading.Lock()
5
6     def predict(self, image_bytes: bytes, filename: str) ->
7     InferenceResult:
8         image = self._decode_image(image_bytes)
9         with self._lock:
10             prediction = self._model.predict(
11                 image, conf=self.confidence, verbose=False
12             )[0]
13             detections = self._extract_detections(prediction)
14             annotated = self._annotate_count(prediction.plot(), len(
15                 detections))
16             # encoda resultado como PNG e retorna InferenceResult
17             ...
```

Listing 3: Estrutura simplificada da classe `CrackDetector`.

O modelo é carregado uma única vez durante o *startup* da aplicação por meio do gerenciador de ciclo de vida (`lifespan`) do FastAPI, eliminando a sobrecarga de I/O e compilação JIT em cada requisição:

```
1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
3     app.state.detector = CrackDetector(MODEL_PATH)
4     yield
```

Listing 4: Carregamento do modelo no startup da aplicação.

5.3 Endpoint de Predição

O endpoint principal POST `/api/predict` (Listagem 5) recebe uma imagem via *multipart/form-data* e retorna um objeto JSON estruturado.

```
1 @app.post("/api/predict", response_model=PredictionResponse)
2 async def predict(request: Request,
3                 file: UploadFile = File(...)) -> PredictionResponse:
4     content = await file.read()
5     detector: CrackDetector = request.app.state.detector
```

```
6     result = await run_in_threadpool(  
7         detector.predict, content, file.filename or "upload.png"  
8     )  
9     return PredictionResponse(  
10         filename=result.filename,  
11         count=result.count,  
12         detections=[PredictionItem(**d.to_dict()) for d in result.  
13         detections],  
14         image_base64=result.to_data_url(),  
15     )
```

Listing 5: Endpoint de predição.

O uso de `run_in_threadpool` delega a inferência (operação bloqueante intensiva em CPU/GPU) a uma *thread* separada, evitando o bloqueio do *event loop* assíncrono do servidor (5).

A resposta segue o esquema `PredictionResponse`:

- **filename**: nome do arquivo enviado;
- **count**: número de rachaduras detectadas;
- **detections**: lista de objetos com `class_id`, `class_name`, `confidence` e `bbox` (x_1, y_1, x_2, y_2);
- **image_base64**: imagem anotada codificada como *data URL* Base64.

5.4 Endpoints Auxiliares

Além do endpoint principal, o serviço expõe:

- `GET /` — interface web HTML para demonstração via navegador;
- `GET /health` — *health check* que retorna `{"status": "ok"}`, útil para orquestradores como Kubernetes.

5.5 Interface Web

A interface web serve como prova de conceito (*proof of concept*) do fluxo completo. Ela permite enviar uma imagem pelo navegador e visualizar a imagem anotada, a contagem de rachaduras e os detalhes das detecções. O frontend é deliberadamente simples: a prioridade da entrega era o pipeline de inferência, não a experiência do usuário.

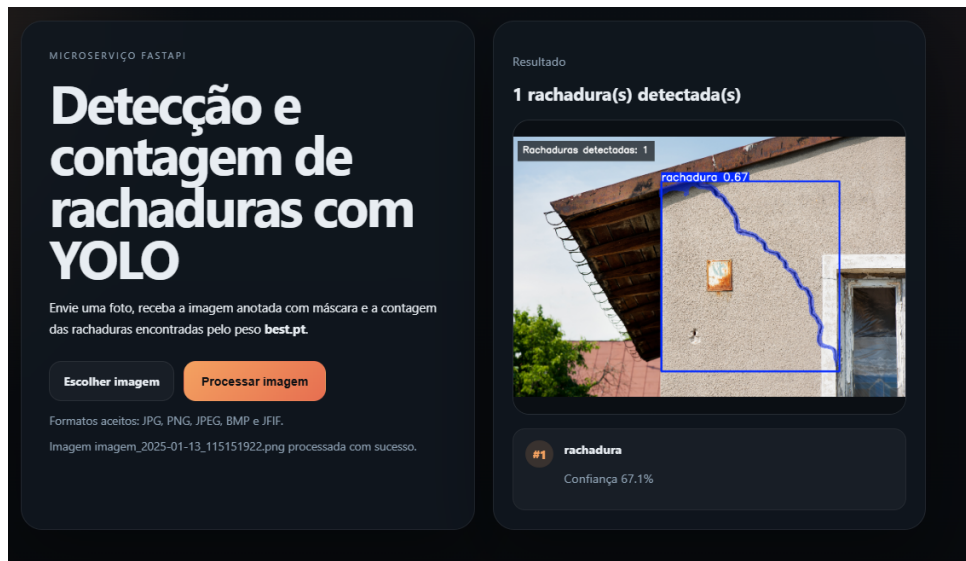


Figura 2: Print da aplicação desenvolvida para o desafio.

6 Empacotamento e Reprodutibilidade

O projeto foi preparado para execução local e em contêiner Docker (16), garantindo reprodutibilidade independentemente do ambiente do desenvolvedor. A containerização encapsula todas as dependências — versão do Python, bibliotecas e pesos do modelo — em uma imagem portátil.

O servidor é iniciado com Uvicorn (17):

```
1 uvicorn.run("app.main:app", host="0.0.0.0", port=8000, reload=False)
```

Listing 6: Inicialização do servidor.

7 Discussão

7.1 Justificativa do Fine-Tuning

A escolha pelo *fine-tuning* em detrimento de um treino completo (*training from scratch*) foi determinada pelo tamanho do conjunto de dados disponível. Treinar um modelo YOLO do zero exigiria, tipicamente, dezenas de milhares de imagens anotadas para convergir a uma solução estável (8). Com uma quantidade moderada de amostras, o modelo de origem (pré-treinado no COCO (12)) fornece representações visuais de baixo nível — detectores de bordas, texturas, gradientes — que são diretamente úteis para a detecção de fissuras, reduzindo a quantidade de exemplos necessários para especialização.

7.2 Limitações

As principais limitações do sistema atual incluem:

1. **Generalização limitada:** o modelo foi treinado em um domínio específico de imagens de rachaduras. Condições de iluminação, ângulo de captura ou tipo de superfície muito diferentes do conjunto de treinamento podem degradar o desempenho.
2. **mAP@50-95 para máscaras:** o valor de 0,276 sugere que as máscaras de segmentação perdem qualidade nos limiares de IoU mais altos, o que pode ser melhorado com mais dados anotados com precisão de borda.

8 Conclusão

Este trabalho documentou o desenvolvimento de um sistema completo de detecção de rachaduras utilizando *fine-tuning* do YOLOv8s-seg. O modelo final atingiu mAP@50 de 0,818 para detecção e 0,728 para segmentação de máscaras no conjunto de teste, com tempo de inferência de aproximadamente 11,7 ms por imagem em GPU.

A estratégia de *fine-tuning* mostrou-se adequada ao contexto: permitiu especializar um modelo com amplo conhecimento visual prévio para o domínio das rachaduras com dados limitados, menor custo computacional e menor risco de instabilidade de treino do que uma abordagem do zero.

A POC em FastAPI transformou o *pipeline* de inferência em um microserviço acessível via HTTP, com boas práticas de engenharia de software: carregamento único do modelo, execução thread-safe, validação de esquemas com Pydantic e separação clara entre camadas de inferência e serviço. O empacotamento em Docker fecha o ciclo do projeto, garantindo reprodutibilidade e facilitando implantação em diferentes ambientes.

Referências

- 1 ZOU, Q. et al. DeepCrack: Learning hierarchical convolutional features for crack detection. *IEEE Transactions on Image Processing*, v. 28, n. 3, p. 1498–1512, 2019.
- 2 YANG, L. et al. Deep concrete inspection using unmanned aerial vehicle towards CSSC database. *arXiv preprint arXiv:1812.05333*, 2018. Disponível em: <<https://arxiv.org/abs/1812.05333>>.
- 3 REDMON, J. et al. You only look once: Unified, real-time object detection. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2016. p. 779–788.
- 4 Ultralytics. *YOLOv8: A New State-of-the-Art Computer Vision Model*. 2023. Disponível em: <<https://github.com/ultralytics/ultralytics>>.
- 5 RAMÍREZ, S. *FastAPI: Fast Web Framework for Building APIs with Python*. 2023. Disponível em: <<https://fastapi.tiangolo.com>>.
- 6 GIRSHICK, R. et al. Rich feature hierarchies for accurate object detection and semantic segmentation. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2014. p. 580–587.
- 7 PAN, S. J.; YANG, Q. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, v. 22, n. 10, p. 1345–1359, 2010.
- 8 YOSINSKI, J. et al. How transferable are features in deep neural networks? In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2014. v. 27. Disponível em: <<https://arxiv.org/pdf/1411.1792>>.
- 9 BOCHKOVSKIY, A.; WANG, C.-Y.; LIAO, H.-Y. M. YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020. Disponível em: <<https://arxiv.org/abs/2004.10934>>.
- 10 FENG, C. et al. Automatic pixel-level crack detection on dam surface using deep convolutional network. *Sensors*, v. 20, n. 7, p. 2069, 2020.
- 11 EVERINGHAM, M. et al. The Pascal Visual Object Classes (VOC) challenge. *International Journal of Computer Vision*, v. 88, n. 2, p. 303–338, 2010.
- 12 LIN, T.-Y. et al. Microsoft COCO: Common objects in context. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. [S.l.: s.n.], 2014. p. 740–755.
- 13 ONNX Contributors. *ONNX: Open Neural Network Exchange*. 2019. Disponível em: <<https://onnx.ai>>.
- 14 COLVIN, S.; Pydantic Contributors. *Pydantic: Data Validation Using Python Type Hints*. 2023. Disponível em: <<https://docs.pydantic.dev>>.
- 15 RONACHER, A.; Pallets Contributors. *Flask: The Python Micro Framework for the Web*. 2023. Disponível em: <<https://flask.palletsprojects.com>>.
- 16 Docker, Inc. *Docker: Accelerated Container Application Development*. 2023. Disponível em: <<https://www.docker.com>>.

17 LANA, ; Encode Contributors. *Uvicorn: An ASGI Web Server for Python*. 2023. Disponível em: <<https://uvicorn.dev/>>.